

UEMF Présence

Défense du projet

Document de préparation à la soutenance orale.
Chaque choix technique est justifié, comparé aux alternatives, et accompagné des questions probables avec leurs réponses modèles.

Étudiant	Mouhssine El Boumshouli
Encadrant	Pr Ahmed El Hilali Alaoui
Formation	EIDIA — UEMF, Semestre 5
Année	2025–2026
Stack	Next.js 16 · React 19 · TypeScript · Prisma 6 · PostgreSQL (Neon) · NextAuth · Vercel
Code source	github.com/MouhssineElBoumshouli/Student-Attendance-System

Table des matières

§ 1	Cadre du projet
§ 2	Pourquoi c'est de la Recherche Opérationnelle
§ 3	Décisions d'architecture (n°1 à n°5)
§ 4	Décisions sur la base de données (n°6 à n°8)
§ 5	Décisions de sécurité (n°9 à n°12)
§ 6	Décisions sur le QR rotatif (n°13 à n°17)
§ 7	Décisions de géolocalisation (n°18 à n°20)
§ 8	Architecture anti-fraude (défense en profondeur)
§ 9	Décisions de modélisation RO (n°21 à n°22)
§ 10	Banque de 20 questions probables du jury
§ 11	Glossaire express

§ 1 — Le problème en deux phrases

À l'UEMF, la prise de présence se fait à l'oral, étudiant par étudiant, via Konosys. Pour une promotion de 30 à 60, ça prend 5 à 15 minutes par séance, c'est sujet à erreur, et un voisin peut répondre « présent » pour un absent sans conséquence.

Notre projet remplace ce processus par un **QR code rotatif** que le professeur affiche en classe et que les étudiants scannent depuis leur smartphone. Le système vérifie automatiquement : identité (compte connecté), validité du QR (cryptographie), présence physique (GPS), unicité de l'appareil (empreinte numérique). Temps de prise : **15 à 30 secondes**.

Pourquoi c'est de la RO : on modélise la prise de présence comme un problème d'optimisation combinatoire — pour chaque couple (étudiant, séance), une variable binaire $x_{ij} \in \{0, 1\}$ indique la présence. On maximise une fonction objectif multi-critère ($Z = \alpha \cdot Z_1 + \beta \cdot Z_2 + \gamma \cdot Z_3$) sous cinq familles de contraintes. C'est le cadre RO canonique : variables / objectif / contraintes.

§ 2 — Pourquoi c'est de la RO

Première question probable du jury : « *Pourquoi est-ce un projet de Recherche Opérationnelle et pas un simple projet d'ingénierie logicielle ?* » La réponse est dans la nature mathématique du problème.

Les trois preuves

- **Variables de décision discrètes.** Pour chaque (étudiant, séance), une décision binaire. Avec $201 \text{ étudiants} \times 100 \text{ séances} = 20\,100$ **variables binaires** à fixer. Cadre classique d'optimisation combinatoire.
- **Fonction objectif quantifiable.** $Z = \alpha \cdot Z_1 + \beta \cdot Z_2 + \gamma \cdot Z_3$ avec $Z_1 =$ couverture, $Z_2 =$ vérification, $Z_3 =$ efficacité. Grandeurs mesurables, comparables.
- **Contraintes mathématiquement explicites.** Cinq familles (unicité, temporel, géographique, crvotographique. appareil) avec formulation algébrique et traduction code vérifiable.

Pourquoi multi-critère ? La « qualité » d'un système de présence n'est pas une grandeur unique. Si on optimisait seulement la rapidité, on supprimerait les vérifications. Si on optimisait seulement la sécurité, ce serait trop lent. La **scalarisation pondérée** est la méthode standard pour ramener un problème multi-objectif à un problème mono-objectif.

§ 3 — Décisions d'architecture

Cinq décisions structurantes : web vs native, framework, langage, UI, hébergement.

DÉCISION N°1

Application web (PWA) au lieu d'une app native iOS/Android

Notre choix : **application web accessible via le navigateur**

POURQUOI CE CHOIX

- 1. Zéro installation.** L'étudiant ouvre une URL. Pas d'App Store.
- 2. Une seule base de code.** Téléphone et portable lancent le même code. On évite trois versions (web + Swift iOS + Kotlin Android).
- 3. Mises à jour instantanées.** Un git push corrige tous les utilisateurs. Pas de cycle App Store.

CE QU'ON A ÉCARTÉ

x App native iOS + Android	Trois codebases, deux cycles release, frais App Store. Surcoût injustifiable.
x React Native / Flutter	Reste à packager comme une app. Caméra et GPS déjà disponibles via API web — bénéfice marginal.
x Application desktop seulement	Le QR doit être scanné par smartphone.

Trade-off accepté : la caméra exige HTTPS. Gratuit sur Vercel (Let's Encrypt) — pas un vrai coût, mais une dépendance à connaître.

QUESTIONS PROBABLES DU JURY

Q. Pourquoi pas une app native ? Ce serait plus fluide.

R. Surcoût pas justifiable. Performances natives pas un goulot (jsQR à 30fps sur entrée de gamme). Pour prod à grande échelle, Capacitor envisageable — pas pour un projet académique.

Q. Et si l'étudiant n'a pas Internet ?

R. L'app a besoin du réseau pour valider le scan auprès du serveur. UEMF dispose Wi-Fi + 4G — scénario sans réseau rare. Buffer offline = piste future.

DÉCISION N°2

Next.js 16 (App Router) comme framework full-stack

Notre choix : **Next.js — framework React pour front + back dans un seul projet**

POURQUOI CE CHOIX

- 1. Un seul projet pour tout.** Front et back cohabitent. Types TypeScript et helpers partagés.
- 2. Rendu hybride.** Chaque page choisit son mode : statique, serveur, ou client. Bonnes performances par cas.
- 3. Écosystème Vercel.** Next.js construit par Vercel. Un git push = un déploiement. HTTPS, CDN, serverless par défaut.

CE QU'ON A ÉCARTÉ

x React + Express séparés	Deux projets à synchroniser, duplication de validation.
x Django / Flask (Python)	Excellents en back, mais intégration UI moderne reste manuelle.
x Ruby on Rails	Productif, mais écosystème JS plus dynamique aujourd'hui.
x SvelteKit / Remix / Nuxt	Concurrents directs. Next.js gagne sur l'écosystème.

QUESTIONS PROBABLES DU JURY

Q. Next.js change vite. Risque pour la maintenance ?

R. Coût réel. On a directement choisi App Router (API officielle). Stabilité long terme vient de React.

Q. Pourquoi pas microservices ?

R. Pour 201 utilisateurs, surdimensionné. Monolithique plus simple à raisonner.

DÉCISION N°3

TypeScript plutôt que JavaScript

Notre choix : **TypeScript — JavaScript avec typage statique vérifié à la compilation**

POURQUOI CE CHOIX

1. **Erreurs attrapées à l'écriture.** Une typo (`session.user.profesorId`) est signalée immédiatement. En JS pur, ce bug atteindrait l'utilisateur.
2. **Auto-complétion intelligente.** L'éditeur propose UNIQUEMENT les champs qui existent.
3. **Refactoring sûr.** Renommer un champ propage l'erreur partout, immédiatement.

CE QU'ON A ÉCARTÉ

x JavaScript pur	Plus rapide au début, coûte cher à long terme. Aucun garde-fou.
x Flow (Facebook)	Concurrent historique, perdu commercialement.
x JSDoc avec annotations	Typage en commentaires — moins puissant et plus pénible.

Trade-off accepté : étape de compilation TS → JS. Next.js le fait automatiquement.

QUESTIONS PROBABLES DU JURY

Q. Le typage à l'exécution n'existe pas en JS. TypeScript ne sécurise rien au runtime ?

R. Exact, et c'est pour ça qu'on utilise **Zod** en complément. TypeScript vérifie à la compilation, Zod à l'exécution.

DÉCISION N°4

React (via Next.js) comme bibliothèque UI

Notre choix : **React 19 — composants réutilisables et mise à jour ciblée**

POURQUOI CE CHOIX

1. **Standard de fait.** Facebook, Netflix, Uber, Airbnb. Documentation, tutoriels — tout existe.
2. **Virtual DOM.** Quand le compteur passe de 7s à 6s, seul ce nombre est redessiné. Indispensable pour temps réel.
3. **Hooks.** Organisent proprement l'état et les effets de bord.

CE QU'ON A ÉCARTÉ

x Vue.js	Excellent, mais écosystème plus petit dans Next.js.
x Angular	Trop lourd pour notre échelle.
x Svelte / Solid	Plus performants. Risque d'apprentissage > gain marginal.
x jQuery / vanilla JS	Pour une UI dynamique, gestion manuelle du DOM ingérable.

DÉCISION N°5

Vercel pour l'hébergement

Notre choix : **Vercel** — plateforme spécialisée Next.js

POURQUOI CE CHOIX

- 1. Zéro configuration.** Connecte le repo GitHub, tout est automatique. Aucune ligne de Docker.
- 2. HTTPS gratuit obligatoire.** Certificat Let's Encrypt. Indispensable pour caméra et GPS.
- 3. Tier gratuit suffisant.** Projet entier à 0 €/mois.

CE QU'ON A ÉCARTÉ

x AWS	Config complexe (VPC, IAM). Payant dès quelques requêtes.
x Heroku	Plus de tier gratuit depuis 2022.
x Self-hosted (VPS)	Coût mensuel + maintenance. Risque opérationnel.
x Netlify	Intégration Next.js moins fluide — Next.js DÉVELOPPÉ par Vercel.

§ 4 — Décisions sur la base de données

Trois décisions : moteur (PostgreSQL), hébergeur (Neon), accès depuis le code (Prisma).

DÉCISION N°6

PostgreSQL comme moteur de base de données

Notre choix : **PostgreSQL — SGBD relationnel ACID, open-source, 30+ ans**

POURQUOI CE CHOIX

- 1. Données fortement relationnelles.** Étudiant → groupe → cours → séance → présence. Cadre exact d'un SGBD relationnel.
- 2. Contraintes d'intégrité automatiques.** UNIQUE(sessionId, studentId) appliqué PAR PostgreSQL, pas par notre code.
- 3. ACID.** Activation séance = UPDATE statut + INSERT 201 lignes. Soit tout réussit, soit tout échoue. Pas de demi-état.

CE QU'ON A ÉCARTÉ

x MySQL / MariaDB	Aussi mature, mais PostgreSQL a meilleur support contraintes complexes, JSONB natif, écosystème serverless plus actif.
x MongoDB (NoSQL)	Nos données relationnelles en documents = duplications ou joins manuels lents. NoSQL pour données semi-structurées.
x SQLite	Fichier local. Utilisé au début. Ne supporte pas plusieurs connexions concurrentes — goulot dès 2 profs simultanés.
x Firebase / Firestore	Vendor lock-in Google. Pricing imprévisible. Pas de SQL — pénalité sur les analytiques.

QUESTIONS PROBABLES DU JURY

Q. Et si vous avez 100 000 étudiants ?

R. PostgreSQL gère des centaines de millions de lignes. Index B-tree sur colonnes critiques en $O(\log n)$. À très grande échelle, AWS RDS Aurora.

Q. Pourquoi pas de cache (Redis) ?

R. Pas nécessaire à notre charge. PostgreSQL + index répond en moins de 10ms. YAGNI.

Q. RGPD ?

R. Données minimales, HTTPS, chiffrement Neon (AES-256), device hash non réversible, suppression cascade par l'admin.

DÉCISION N°7

Neon Cloud comme hébergeur PostgreSQL

Notre choix : **Neon — PostgreSQL géré, serverless, offre gratuite**

POURQUOI CE CHOIX

- 1. PostgreSQL standard.** Aucun lock-in. On peut migrer vers AWS RDS, Supabase, self-hosted sans toucher au code.
- 2. Serverless = gratuit à notre échelle.** Pas de machine 24/7. Le service se réveille à la demande.
- 3. Branches de BDD.** Comme Git pour la base. Tester des migrations sans casser la prod — unique à Neon.

CE QU'ON A ÉCARTÉ

x Supabase	Concurrent direct, mais pousse son écosystème (auth, realtime) qu'on n'utilise pas.
x Railway / Render	Tier gratuit limité dans le temps ou heures CPU.
x AWS RDS / Aurora	Industriel, payant dès le premier jour. Surdimensionné.
x Self-hosted (Docker + VPS)	Coût mensuel + maintenance (sauvegardes, monitoring, patches).

QUESTIONS PROBABLES DU JURY

Q. Le serverless ajoute une latence de cold start. Problème ?

R. Première requête après inactivité ~1s. Invisible pour le prof. Pour les scans en rafale, les suivantes sont instantanées.

DÉCISION N°8

Prisma 6 comme ORM

Notre choix : **Prisma — ORM qui traduit notre code TypeScript en SQL**

POURQUOI CE CHOIX

1. **Schéma déclaratif unique.** schema.prisma = source de vérité. Prisma génère TOUT : migrations, types, requêtes type-safe.
2. **Requêtes type-safe.** Une typo dans un nom de colonne attrapée par TypeScript.
3. **Migrations versionnées.** Reproductible sur tous les environnements.
4. **Pas de SQL injection.** Paramètres préparés systématiques.

CE QU'ON A ÉCARTÉ

x SQL brut + pg (driver)	Plus performant, mais perd la sécurité du typage et la productivité des migrations.
x Drizzle ORM	Concurrent récent, plus léger. Bonne alternative — Prisma reste majoritaire.
x TypeORM	Plus ancien, moins type-safe. En désuétude.
x Sequelize	ORM JS historique. Pas conçu autour de TypeScript.

QUESTIONS PROBABLES DU JURY

Q. Les ORM sont lents. Mesuré ?

R. Pour 99% des requêtes simples, Prisma génère du SQL aussi efficace qu'à la main. Pour les requêtes très complexes, on peut basculer en `prisma.$queryRaw`.

§ 5 — Décisions de sécurité

Quatre décisions critiques : NextAuth, JWT, bcrypt, RBAC.

DÉCISION N°9

NextAuth.js pour gérer les sessions

Notre choix : **NextAuth — bibliothèque d'auth standard de Next.js**

POURQUOI CE CHOIX

- 1. **On ne réinvente pas la sécurité.** Auth custom = source majeure de vulnérabilités. NextAuth audité par milliers de projets.
- 2. **Cookies signés gratuits.** JWT en cookie httpOnly, Secure, SameSite=Lax — protection XSS et CSRF par défaut.
- 3. **Extensible.** Sign in with Google UEMF = une ligne de config.

CE QU'ON A ÉCARTÉ

x Système d'auth custom	Bombe à retardement. On oublie toujours un détail.
x Auth0 / Clerk	Services payants au-delà d'un seuil. Excellents pour startups, dépendance inutile pour académique.
x Firebase Auth	Lock-in Google. Migration des comptes pénible.

DÉCISION N°10

JWT (sessions stateless) plutôt que sessions serveur

Notre choix : **JWT — token signé contenant les infos utilisateur, en cookie**

POURQUOI CE CHOIX

1. **Stateless = scalable.** Le serveur ne mémorise rien entre requêtes. Si on multiplie les serveurs, pas de mémoire à partager.
2. **Signature cryptographique.** Le serveur vérifie la signature à chaque requête. Modifier son rôle = casser la signature.
3. **Expiration automatique.** 24h. Pas de job de nettoyage.

CE QU'ON A ÉCARTÉ

x Sessions serveur (table SQL) Marche, mais une requête BDD par appel.

x Sessions en mémoire (Redis) Plus rapide, mais demande Redis en plus.

x JWT en localStorage Mauvaise idée : XSS peut lire. Cookie httpOnly l'empêche.

QUESTIONS PROBABLES DU JURY

Q. Si le JWT est volé ? Vous ne pouvez pas révoquer.

R. Critique valide. Mitigation : cookie httpOnly (bloque XSS), expiration 24h, et tournage de NEXTAUTH_SECRET en cas de compromission — invalide tous les JWT instantanément.

Q. Pourquoi 24h et pas plus court ?

R. Compromis UX vs sécurité. 1h = friction inutile. 30j = fenêtre dangereuse. 24h standard.

DÉCISION N°11

bcrypt pour hasher les mots de passe

Notre choix : **bcrypt avec coût 10 — fonction de hash volontairement lente (100ms)**

POURQUOI CE CHOIX

1. **Volontairement lent.** Tester 10M mots de passe = 12 jours. Brute-force impossible en pratique.
2. **Salt automatique intégré.** Deux utilisateurs avec même mot de passe = hashes différents.
3. **Coût ajustable.** 10 → 12 → 14 quand les CPU s'accélèrent.

CE QU'ON A ÉCARTÉ

x MD5 / SHA-1	Trop rapide. Brute-force trivial.
x SHA-256 / SHA-512	Idem : rapide. Pour signer (HMAC) oui, pour mots de passe non.
x Argon2	Plus récent, techniquement supérieur. Pour un nouveau projet — bcrypt reste très acceptable.
x scrypt	Alternative à bcrypt, plus de mémoire. Adoption moindre.

QUESTIONS PROBABLES DU JURY

Q. Le mot de passe par défaut password123 est faible.

R. Vrai, intentionnel pour la démo. En prod : mot de passe aléatoire à chaque création + obligation de changement à la 1re connexion.

DÉCISION N°12

Contrôle d'accès par rôle (RBAC) côté serveur

Notre choix : **RBAC strict — chaque endpoint vérifie le rôle, helper centralisé**

POURQUOI CE CHOIX

1. **Défense en profondeur.** Même si le front cache un bouton, l'API refuse indépendamment. JAMAIS de confiance au client.
2. **Centralisé = uniforme.** Une fonction implémente la vérif.
3. **Vérification de propriété.** Un prof peut activer SES séances, pas celles d'un autre prof.

CE QU'ON A ÉCARTÉ

x Vérif seulement côté frontend	Erreur classique. Frontend modifiable. N'importe qui peut requêter l'API directement.
x Vérif ad-hoc dans chaque route	Répétitif et fragile. On finit par oublier une route.
x ABAC (attribute-based)	Plus granulaire, surkill pour nos trois rôles.

QUESTIONS PROBABLES DU JURY

Q. Démontrez que je ne peux pas marquer un autre étudiant présent.

R. L'API exige STUDENT et IGNORE le studentId du body — lit l'identité depuis le JWT.
Démon : fetch avec studentId='absent' dans le body — c'est MOI qui suis marqué présent.

§ 6 — Décisions sur le QR rotatif

Cinq décisions imbriquées : algo crypto, durée rotation, longueur token, mode polling, stabilité visuelle.

DÉCISION N°13

HMAC-SHA256 comme schéma de génération de tokens

Notre choix : **HMAC-SHA256 — inspiré de TOTP (RFC 6238, Google Authenticator)**

POURQUOI CE CHOIX

- 1. Déterministe côté serveur uniquement.** À partir du secret et de l'indice k, le serveur recalcule le token. Le client n'a pas le secret — ne peut prédire les tokens futurs.
- 2. Cryptographiquement sûr.** SHA-256 = fonction hash industrielle, sans collision connue. HMAC ajoute l'authenticité.
- 3. Coût négligeable.** Microsecondes. Milliers de scans/s sans difficulté.

CE QU'ON A ÉCARTÉ

x UUID aléatoire à chaque cycle	Oblige le serveur à STOCKER le token. HMAC = calculable à la demande. Pas d'état.
x RSA / signature asymétrique	100x plus coûteux. RSA utile quand plusieurs parties vérifient sans partager. Ici, une seule partie.
x SHA-256 seul (sans HMAC)	Sans clé secrète, n'importe qui calcule le hash.
x Compteur incrémental	Trivialement devinable. Aucune sécurité.

QUESTIONS PROBABLES DU JURY

Q. Pourquoi pas TOTP exactement comme Google Authenticator ?

R. Notre schéma est très proche. TOTP = 6 chiffres lisibles humain. Notre QR lu par caméra = on se permet 8 hex (plus d'entropie).

DÉCISION N°14

Fenêtre de rotation de 10 secondes

Notre choix : $\Delta = 10$ secondes, validité fenêtre courante + précédente

POURQUOI CE CHOIX

1. **Trop court ($\leq 3s$)** : les étudiants n'ont pas le temps. Échecs, frustration.
2. **Trop long ($\geq 60s$)** : photo a le temps de circuler en messagerie. Fraude.
3. **10s** : confortable pour scanner, trop court pour partager.

CE QU'ON A ÉCARTÉ

x 5 secondes	Trop serré. Premiers tests : scans rataient (latence GPS+caméra).
x 20 secondes (valeur initiale)	Notre choix de départ. Réduit à 10s pour resserrer.
x QR statique pour toute la séance	Impossible à protéger contre la photo.

QUESTIONS PROBABLES DU JURY

Q. Pourquoi vous acceptez aussi la fenêtre précédente ?

R. Pour gérer les scans À CHEVAL sur la limite. L'étudiant scanne à la 9,8e seconde : capture, GPS, requête réseau... arrive après le changement. Sans tolérance, refusé.

DÉCISION N°15

Token de 8 caractères hexadécimaux (32 bits)

Notre choix : **8 premiers hex de HMAC-SHA256 — 32 bits = 4,3 milliards de possibilités**

POURQUOI CE CHOIX

1. **Suffisant contre brute-force réseau.** Fenêtre 10s, rate-limit : brute-force impossible.
2. **QR plus rapide à scanner.** Moins dense = caméra décode plus vite.
3. **Lisibilité de debug.** 8 caractères tiennent dans les logs.

CE QU'ON A ÉCARTÉ

x 16 caractères hex (64 bits) Surdimensionné. Protège contre menace inexistante.

x 4 caractères hex (16 bits) Insuffisant : 65 536 possibilités. Brute-force trivial.

x 6 chiffres décimaux (comme TOTP) 1M possibilités — limite acceptable, moins solide.

QUESTIONS PROBABLES DU JURY

Q. 32 bits semble peu pour de la cryptographie moderne.

R. En crypto de transit (TLS), 256 bits — autre contexte. Ici : (1) token jetable expire en 10s, (2) chaque essai = requête HTTP rate-limitée. À 1000 req/s, deviner 2^{32} = 50 jours. Impossible.

DÉCISION N°16

Polling intelligent plutôt que WebSocket/SSE

Notre choix : **polling planifié 200ms après l'expiration de la fenêtre**

POURQUOI CE CHOIX

1. **Un seul fetch par fenêtre.** 6/minute au lieu de 30. Économie 80% bande passante.
2. **Affichage fluide.** Compte à rebours dérivé de (expiresAt - Date.now()).
3. **Pas d'infra persistante.** WebSocket/SSE = connexion ouverte, complique le serverless.

CE QU'ON A ÉCARTÉ

x WebSocket bidirectionnel Surdimensionné pour flux server vers client. Mal supporté serverless.

x SSE (Server-Sent Events) Plus simple, même problème de connexion persistante.

x Polling à intervalle fixe (2s) Version initiale. Trafic inutile + clignotement (bug corrigé).

QUESTIONS PROBABLES DU JURY

Q. Pourquoi pas en temps réel ?

R. Moderne n'est pas un critère. Nos données changent toutes les 10s par construction — rien à pousser en sub-seconde.

DÉCISION N°17

Timestamp windowStart dans le payload, pas Date.now()

Notre choix : **windowStart = $k \times \Delta$ (début de fenêtre, fixe 10s)**

POURQUOI CE CHOIX

1. **QR visuellement stable.** Si le payload change à chaque seconde, le canvas redessine, ça clignote. Avec windowStart, payload identique sur toute la fenêtre — QR dessiné une seule fois.
2. **Validation reste correcte.** Le timestamp sert au contrôle de dérive ($\pm 60s$). windowStart au plus 10s plus vieux — dans la tolérance.

QUESTIONS PROBABLES DU JURY

Q. Pourquoi avoir mentionné ce bug dans le rapport ?

R. Montre qu'on a (1) testé avec vrais utilisateurs, (2) identifié la cause racine, (3) corrigé proprement. Trace d'un cycle de dev honnête.

§ 7 — Décisions de géolocalisation

Trois décisions : formule de calcul, configurabilité du rayon, statut « non vérifié ».

DÉCISION N°18

Formule de Haversine pour calculer la distance

Notre choix : **Haversine — distance entre deux points GPS sur une sphère**

POURQUOI CE CHOIX

1. **Modèle sphérique simple et correct.** Erreur sub-métrique à l'échelle d'un campus.
2. **Pas de dépendance externe.** ~10 lignes de code. Gratuit, pas de rate limit.
3. **Standard universel.** Tout le monde la connaît.

CE QU'ON A ÉCARTÉ

x Théorème de Pythagore	Suppose un plan plat. Pour campus l'erreur est négligeable, mais Haversine aussi simple ET correct par construction.
x Formule de Vincenty (ellipsoïde)	Plus précise, surdimensionnée pour quelques centaines de mètres.
x Google Maps Distance API	Externalise un calcul trivial. Dépendance, coût, latence.

DÉCISION N°19

Rayon de geofence configurable par salle

Notre choix : **Chaque salle a son Room.radius — 60m Labo, 80m Salle, 150m amphis**

POURQUOI CE CHOIX

1. **Précision GPS varie.** Plein air $\pm 5m$, intérieur $\pm 10-30m$, grand amphi $\pm 50m$. Un rayon unique exclurait ou inclurait à tort.
2. **Taille physique des salles.** Labo 5x6m mérite 60m. Amphi 30x40m mérite 150m.
3. **Réglable par l'admin.** Sans redéploiement.

QUESTIONS PROBABLES DU JURY

Q. Et si quelqu'un usurpe sa position GPS ?

R. Vrai sur téléphone rooté. Mitigations : (1) fraude active compétente, (2) la couche fingerprint détecte, (3) marqué non vérifié et arbitré par le prof.

Q. Pourquoi pas vérifier le Wi-Fi campus à la place ?

R. Complémentaire — mais ne dit pas dans quelle salle. GPS donne la position fine.

DÉCISION N°20

Marquer « non vérifié » plutôt que refuser le scan

Notre choix : **GPS hors rayon ou absent → présence marquée non vérifiée, prof décide**

POURQUOI CE CHOIX

1. **GPS imprécis en intérieur.** Étudiant près d'un mur peut dériver 30m. Rejet automatique = injuste.
2. **Le prof a le contexte que le système n'a pas.** Il voit l'étudiant en classe. 1 clic.
3. **L'information est conservée.** Trace gardée — audit possible.

CE QU'ON A ÉCARTÉ

- | | |
|---------------------------------------|---|
| x Rejet automatique hors rayon | Trop binaire. Faux négatifs qui ruinent la confiance. |
| x Ignorer le GPS complètement | Couche anti-fraude disparaît. |

QUESTIONS PROBABLES DU JURY

Q. Cela donne l'impression que le système doute des étudiants.

R. Étudiant lambda en classe verra GPS OK 99% du temps. Non vérifié concerne surtout refus permission GPS, erreur exceptionnelle, cas suspect réel.

§ 8 — Architecture anti-fraude

Quatre couches indépendantes. L'attaquant doit contourner les QUATRE en même temps pour réussir une fraude sans laisser de trace.

Défense en profondeur. Plutôt qu'un mur unique « parfait », on superpose plusieurs lignes de défense de natures différentes. Un attaquant qui passe l'une est arrêté par la suivante.

Attaques et neutralisations

Attaque	Neutralisée par
Photo du QR → ami absent	C2 — rotation 10s, photo expirée à l'arrivée
Appel vidéo → scan à distance	C3 — GPS hors rayon, marqué non vérifié
Un téléphone pour deux comptes	C5 — empreinte d'appareil identique détectée
Bypass API direct (POST faux ID)	JWT — studentId lu de la session, pas du body
Brute-force du token	C4 — 2^{32} possibilités, fenêtre 10s, rate limit
Manipulation horloge téléphone	C2 — fenêtre calculée côté serveur, $\pm 60s$

Aucune SEULE couche ne résiste à toutes ces attaques. Combinées, l'attaquant doit : prédire un token en 10s, usurper GPS, avoir un téléphone unique, être connecté avec un compte légitime. Négligeable pour fraude opportuniste.

Q. Un étudiant motivé peut quand même contourner. Pourquoi alors ?

R. Vrai. Mais : (1) profil rare, (2) la barre d'effort rend la fraude PRÉMÉDITÉE (plus opportuniste), (3) on conserve les traces — audit possible. Le système n'est pas parfaitement incassable, il est ÉCONOMIQUEMENT DÉFAVORABLE à la fraude.

§ 9 — Décisions de modélisation RO

$S = \{\text{séances}\}$, $E = \{\text{étudiants}\}$, $T = \{\text{fenêtres QR}, \Delta=10s\}$, $R = \{\text{salles}\}$. Variables : x_{ij} présence, L_{ij} retard, v_{ij} vérifié — toutes binaires.

Objectif : Maximiser $Z = \alpha \cdot Z_1 + \beta \cdot Z_2 + \gamma \cdot Z_3$ ($\alpha=0,4$, $\beta=\gamma=0,3$).

Contraintes : C1 unicité, C2 temporel, C3 géographique (Haversine), C4 cryptographique (HMAC), C5 appareil.

DÉCISION N°21

Variables binaires plutôt que continues

Notre choix : $x_{ij} \in \{0, 1\}$ — présent ou pas, pas un degré

POURQUOI CE CHOIX

- 1. Réalité métier binaire.** Un étudiant est présent ou absent au sens administratif. Pas de 70%.
- 2. Décision claire.** Statut PRESENT/ABSENT déclenche des actions distinctes (assiduité, alertes).

CE QU'ON A ÉCARTÉ

x $x_{ij} \in [0, 1]$ (probabilité) Aurait du sens si on agrégeait plusieurs sources d'évidence — surdimensionné.

x Variable multinomiale Reflète notre cas : statut = PRESENT, ABSENT, LATE, EXCUSED — encodé en plusieurs variables binaires.

DÉCISION N°22

Scalarisation pondérée plutôt que front de Pareto

Notre choix : **combinaison convexe** $Z = \alpha \cdot Z_1 + \beta \cdot Z_2 + \gamma \cdot Z_3$ avec $\alpha + \beta + \gamma = 1$

POURQUOI CE CHOIX

1. **Une seule fonction à interpréter.** Z = score entre 0 et 1, facile à communiquer.
2. **Préférences explicites.** α, β, γ encodent les préférences du décideur.
3. **Calculable en temps réel.** Pas besoin d'un solveur à chaque scan.

CE QU'ON A ÉCARTÉ

- x Front de Pareto** Utile quand on ne connaît pas ses préférences. Ici, on les connaît.
- x Ordre lexicographique** Marche si un objectif domine absolument. Notre cas plus nuancé.
- x Min-max** Approche équité — pas adapté.

QUESTIONS PROBABLES DU JURY

Q. Comment vous avez choisi $\alpha=0,4, \beta=\gamma=0,3$?

R. Calibrage métier initial. $\alpha=0,4$ (couverture) = priorité n°1. $\beta=\gamma=0,3$ (vérification et efficacité) à égalité, importants mais secondaires. CONFIGURABLES — calibrage empirique après usage prod. Analyse de sensibilité classique en RO.

§ 10 — Banque de 20 questions probables

Vingt questions courantes qu'un jury RO posera. Pour chacune, une réponse modèle à mémoriser — adaptée à votre projet.

Q1 - CADRAGE

En quoi votre projet est-il un projet de Recherche Opérationnelle et pas un simple projet d'informatique ?

Trois preuves : **(1)** 20 100 variables binaires de décision (201 x ~100 séances) — optimisation combinatoire. **(2)** fonction objectif quantifiable $Z = \alpha \cdot Z_1 + \beta \cdot Z_2 + \gamma \cdot Z_3$. **(3)** cinq familles de contraintes mathématiquement explicites. Structure canonique RO.

Q2 - BASE DE DONNÉES

Pourquoi PostgreSQL et pas MongoDB ?

Données fortement relationnelles (étudiant → groupe → cours → séance → présence). En documents JSON : duplications ou joins manuels lents. PostgreSQL conçu pour ce cas : contraintes UNIQUE et FK garanties, JOIN performants.

Q3 - CRYPTOGRAPHIE

Pourquoi HMAC-SHA256 et pas un simple UUID aléatoire ?

UUID = oblige le serveur à STOCKER le token. HMAC = CALCULABLE à la demande à partir du secret et de l'indice. Pas d'état supplémentaire. HMAC garantit aussi l'authenticité.

Q4 - ROTATION

Pourquoi 10 secondes et pas 5 ou 60 ?

5s = trop court (latence GPS+caméra). 60s = photo a le temps de circuler. 10s = confortable pour scanner, trop court pour partager. Itération initiale à 20s.

Q5 - TOKEN

32 bits semble peu pour de la cryptographie. Justifiez.

Le facteur limitant n'est pas la longueur mais le DÉBIT D'ATTAQUE. Chaque essai = requête HTTP rate-limitée. $2^{32} = 4,3$ milliards à 1000 req/s → 50 jours. Token expire en 10s. Impossible.

Q6 - GPS

Le GPS peut être falsifié. Quelle est la vraie sécurité ?

Vrai sur téléphone rooté. Mais : (1) fraude PRÉMÉDITÉE, pas « réponse présent », (2) autres couches actives (fingerprint), (3) GPS suspect → marqué non vérifié, prof arbitre. Système ÉCONOMIQUEMENT DÉFAVORABLE à la fraude.

Q7 - SÉCURITÉ

Comment empêchez-vous un étudiant de marquer un autre présent via l'API directement ?

L'API exige STUDENT et IGNORE le studentId du body. Identité lue depuis le JWT signé.
Démon : fetch avec studentId='absent' dans le body — le serveur me marque MOI, pas l'absent.

Q8 - SCALABILITÉ

Combien d'étudiants votre système peut supporter ?

Aujourd'hui 201, scalable à des dizaines de milliers. PostgreSQL gère centaines de millions de lignes, requêtes indexées en $O(\log n)$. Vercel scale horizontalement, Neon verticalement.

Q9 - MULTI-CRITÈRE

Justifiez le choix des poids $\alpha=0,4$, $\beta=\gamma=0,3$.

Calibrage métier initial. $\alpha=0,4$ (couverture) = priorité n°1. $\beta=\gamma=0,3$ (vérification et efficacité) à égalité. Poids CONFIGURABLES. Calibrage empirique après usage prod.

Q10 - STACK

Pourquoi Next.js et pas une autre stack ?

(1) Front+back dans un seul projet — pas de synchro. (2) Déploiement Vercel trivial avec HTTPS gratuit. (3) Écosystème React le plus mature.

Q11 - MAINTENANCE

Qu'est-ce qui rend votre code maintenable sur 5 ans ?

TypeScript = cohérence types, refactoring sûr. Prisma = schéma BDD centralisé. API REST claire avec Zod. Composants UI isolés. GitHub avec historique.

Q12 - RGPD

RGPD ? Vous collectez la position GPS des étudiants.

(1) Données minimales (email, nom, ID, GPS pendant séance). (2) HTTPS en transit. (3) Chiffrement Neon AES-256. (4) Device hash non réversible (SHA-256). (5) Suppression cascade par l'admin.

Q13 - PANNE

Que se passe-t-il si Neon (la BDD) tombe pendant un cours ?

Erreur 500 aux scans. Le prof voit immédiatement. Mitigations : Neon SLA 99,9%, sauvegardes horaires. En cas de panne, le prof bascule en mode manuel.

Q14 - ERGONOMIE

Et si un étudiant n'a pas de smartphone ?

Le prof peut le marquer présent manuellement depuis son tableau de bord — fonctionnalité testée. Le système ACCÉLÈRE le cas standard, sans bloquer les cas particuliers.

Q15 - ARCHITECTURE

Pourquoi pas une architecture en microservices ?

Pour 201 utilisateurs, surdimensionné. Microservices = complexité opérationnelle pour bénéfice nul à notre échelle. Monolithique plus simple à raisonner.

Q16 - TESTS

Comment vous testez le système ?

Tests manuels sur flux critiques. TypeScript = premier filet. Validation Zod sur tous les endpoints. Build CI/CD à chaque push. Roadmap V2 : Playwright pour intégration, Vitest pour unitaires.

Q17 - COÛT

Combien ça coûte à exploiter par an ?

Zéro euro. Vercel + Neon + Let's Encrypt + GitHub tous gratuits sous nos seuils. Au pire ~20€/mois si on dépasse.

Q18 - INTEROP

Comment intégrer ça avec Konosys ?

Export CSV (RFC 4180, importable Konosys). Si Konosys expose une API REST, on écrirait un connecteur pour pousser les présences.

Q19 - FUTUR

Quelles améliorations à court terme ?

(1) Alertes profs si seuil d'absences. (2) Analyse temporelle d'assiduité par cours. (3) Mode hors-ligne. (4) Notification au prof si scan non vérifié. (5) Intégration Konosys via API.

Q20 - DÉMO

Faites-moi une démo en direct.

Plan 3 min : (1) connexion Pr Ahmed. (2) activation séance RO — 201 ABSENT pré-crées. (3) téléphone : scan QR. (4) dashboard prof se met à jour, étudiant à PRESENT en ~5s. (5) override manuel. (6) terminer, export CSV.

§ 11 — Glossaire express

Trente termes pour rafraîchir la mémoire pendant la soutenance.

Terme	Définition courte
ACID	Atomicité, Cohérence, Isolation, Durabilité — garanties des transactions BDD.
ADMIN/PROFESSOR/STUDENT	Les trois rôles utilisateurs.
App Router	Le système de routage moderne de Next.js.
bcrypt	Hash de mot de passe volontairement coûteux.
B-tree	Index BDD, lookups en $O(\log n)$.
CSV (RFC 4180)	Format texte tabulaire, champs entre guillemets.
Défense en profondeur	Plusieurs couches indépendantes.
Fingerprint	Empreinte numérique d'un appareil (UA + résolution + langue → SHA-256).
Haversine	Formule de distance géodésique sur une sphère.
HMAC-SHA256	Code d'authentification basé sur SHA-256.
HTTPS	HTTP chiffré par TLS — obligatoire pour caméra + GPS.
JWT	JSON Web Token — token d'auth signé.
NextAuth	Bibliothèque d'authentification Next.js.
Optimisation combinatoire	Optimisation sur variables discrètes.
ORM	Object-Relational Mapper — pont code vers tables SQL.
Pareto	Solutions non-dominées en multi-objectif.
Polling	Client interroge le serveur à intervalle régulier.
PostgreSQL	SGBD relationnel ACID open-source.
Prisma	Notre ORM TypeScript — schéma déclaratif, type-safe.
RBAC	Role-Based Access Control.
React	Bibliothèque UI à composants, mise à jour ciblée.
Scalarisation	Méthode multi-critère vers scalaire unique.
Serverless	Architecture sans serveur persistant.
TOTP (RFC 6238)	Time-based One-Time Password — Google Authenticator.

Terme	Définition courte
TypeScript	JavaScript avec typage statique.
UNIQUE(a, b)	Contrainte SQL interdisant doublons (a, b).
Vercel	Plateforme d'hébergement, auteurs de Next.js.
WebSocket	Connexion bidirectionnelle persistante (écartée).
Zod	Validation runtime des données entrantes.
$Z = \alpha \cdot Z_1 + \beta \cdot Z_2 + \gamma \cdot Z_3$	Fonction objectif scalaire — fiabilité globale.

Code source : github.com/MouhssineElBoumshouli/Student-Attendance-System

Stack : Next.js 16 · React 19 · TypeScript · Prisma 6 · PostgreSQL (Neon) · NextAuth · Tailwind v4 · Vercel